Transformer-Based Portfolio System - Design Document

Introduction

This project implements an end-to-end quantitative portfolio management system that combines transformer-based machine learning with mean-variance optimization to construct market-neutral equity portfolios. The system predicts both returns and volatility for individual stocks, then uses these predictions to build optimal portfolios with explicit risk control.

Overview

The system operates through four integrated components:

1. **Feature Engineering**: Constructs 41 technical, fundamental, and market features for each stock-date observation
2. **Market Regime Detection**: Hidden Markov Model identifies latent market states (bull/bear/transitional)
3. **Neural Forward Model**: Transformer neural network predicts 20-day forward returns and daily volatility
4. **Portfolio Optimization**: Mean-variance optimization constructs market-neutral portfolios with transaction cost modeling

The architecture achieves an annualized portfolio returns of 4.46% and Sharpe ratio of 1.14 on out of sample test data, demonstrating strong predictive power and effective translation to risk-adjusted profits while maintaining market neutrality.

Data Pipeline and Train/Eval/Test Split

The system uses a strict temporal train/validation/test split to prevent look-ahead bias:

Training Set (2010-01-01 to 2021-01-01)

- **Purpose**: Learn relationships between features and forward returns/volatility
- **Samples**: ~1.29M stock-date observations
- **Usage**: Train transformer model weights, fit normalization parameters
- **Expected IC**: 0.18-0.20 (with some overfitting expected)

Evaluation Set (2022-01-01 to 2023-07-01)

- **Purpose**: Model checkpoint selection via lowest NLL eval loss
- **Samples**: ~250K observations

- **Usage**: Select best checkpoint by minimum eval loss, validate generalization
- **Selection Criterion**: Lowest eval loss (magnitude accuracy over ranking)

Test Set (2024-07-01 to 2026-01-01)

- **Purpose**: Final unbiased performance evaluation
- **Samples**: ~186K observations
- **Usage**: Simulate live deployment, no parameter changes allowed
- **Results**: Return=+6.78% (18 months), Annualized=4.46%, Sharpe=1.14 (0.5% RFR), Max DD=2.75%
- **Configuration**: max_scalar=16, max_leverage=2, risk_free_rate=0.005, 20-day rebalancing, market-neutral

The temporal structure ensures that all predictions use only information available at the prediction time, preventing subtle forms of data leakage that would inflate backtested performance.

Feature Engineering

The system constructs a comprehensive feature set combining technical indicators, fundamental ratios, and market observations. All features are calculated using only point-in-time data to prevent look-ahead bias.

Technical Features (via `equities/technicals/`)

Price Momentum (Multiple Timeframes)

- Rate of Change (ROC): 20, 60, 120, 250-day periods
- Aligned with 20-day prediction horizon for better generalization
- Captures medium to long-term trends (short-term noise filtered)
- Normalized to handle different price scales

Relative Performance

- Relative Return vs. Market: 20, 60, 250-day periods
- Measures stock-specific performance independent of market moves
- Critical for market-neutral strategies

Beta Estimation

- Rolling Beta vs. S&P 500: 60, 250-day windows
- Quantifies systematic risk exposure
- Used for position sizing and risk management

Volatility Indicators

- ATR (Average True Range): 14-day period
- RSI (Relative Strength Index): 14-day period
- Identifies overbought/oversold conditions

Drawdown Metrics

- Maximum Drawdown: 20, 60, 120, 250-day lookbacks
- Measures downside risk and recovery patterns

- Early warning signal for deteriorating stocks

Volume Analysis

- Log Volume Normal Scores: 20, 60, 120, 250-day windows
- Detects abnormal trading activity
- Leading indicator for price moves

Price Distribution Scores

- Log Close Normal Scores: 20, 60, 120, 250-day windows
- Standardized position within recent price range
- Mean-reversion signal

Fundamental Features (via `equities/fundamentals/`)

Valuation Ratios

- E/P (Earnings-to-Price): Value investing signal
- S/P (Sales-to-Price): Revenue-based valuation
- CF/P (Cash Flow-to-Price): Quality of earnings
- B/P (Book-to-Price): Asset-based valuation

Financial Health

- D/A (Debt-to-Assets): Leverage and solvency
- Operating Margin: Profitability efficiency
- ROE (Return on Equity): Shareholder value creation

Growth Metrics

- Revenue Growth: YoY quarterly growth rate
- Market Cap: Size factor exposure

Data Quality

- FUND_NA, TECH_NA: Binary indicators for missing data
- Allows model to learn missingness patterns

All fundamental data uses quarterly filings with proper alignment to announcement dates (not report period end dates) to prevent look-ahead bias.

Market Features (via `markets/`)

Market Observations

- Term Structure Difference: VIX futures term structure slope
- 20-Day Log Return: Recent market momentum
- 20-Day Log Return Std: Recent market volatility
- Yield Slope: 10Y-2Y Treasury spread (recession indicator)

Regime Probabilities (from HMM)

- regime_0, regime_1, regime_2: Probability distribution over 3 hidden states
- Captures bull/bear/transitional market environments

- Allows model to adapt predictions to market conditions

Total Feature Count: 41 features per stock-date observation

- 24 technical indicators (momentum, volatility, volume, drawdown)
- 10 fundamental ratios (valuation, profitability, growth)
- 4 market observations (returns, volatility, term structure, yield curve)
- 3 regime probabilities (HMM-based market states)

Features are stored in a 60-day temporal window (lookback period), allowing the transformer model to capture time-dependent patterns and feature interactions across multiple timeframes.

Hidden Markov Model for Market Regimes

The Hidden Markov Model (HMM) identifies latent market states that influence all stocks simultaneously.

Training Process:

1. **Input Features**: Market observations (20-day returns, volatility, yield slope, term structure)
2. **Algorithm**: Baum-Welch (Expectation-Maximization) with 50 iterations
3. **States**: 3 hidden states with learned transition probabilities
4. **Output**: Probability distribution over states for each trading day

Implementation (`markets/regime/`):

- `model.py`: Orchestrates regime detection pipeline
- `hmm.py`: Core HMM implementation with forward-backward algorithm

The regime probabilities serve as additional features to the forward model, allowing predictions to adapt based on market environment. This captures the empirical observation that factor performance varies across regimes (e.g., momentum stronger in bull markets, value stronger in bear markets).

Transformer Forward Model

The forward model is a transformer-based neural network that predicts both 20-day forward returns and daily volatility for each stock. This joint prediction enables sophisticated portfolio construction with explicit uncertainty quantification.

Architecture (`models/model.py`):

- **Input**: 60-day window × 41 features = temporal sequence
- **Input Layer**: Linear projection from 41 features to hidden dimensions
- **Attention Mechanism**:
  - 1 transformer encoder layer
  - 2 attention heads
  - Learns which historical time steps and features are most predictive
  - Captures non-linear feature interactions across time
- **Hidden Layers**: 16 hidden units with layer normalization
- **Feed-Forward**: 16 → 64 → 16 dimensions within transformer layer
- **Output Layer**: Linear projection from 16 hidden to 2 targets

- **Total Parameters**: ~3,800 trainable parameters (compact architecture)
- **Output**: 2 values per prediction
  - $\hat{\mu}$: Predicted 20-day forward return
  - $\hat{\sigma}$: Predicted daily volatility over the forward period

Training Process (`scripts/train.py`):

Loss Function: Negative Log-Likelihood (NLL) with lambda_mu weighting

```
L = λ_μ × [(y - μ)² / (2σ²)] + log(σ)
```

Where:

- λ_μ: 1 (equal weighting between return accuracy and uncertainty calibration)
- loss_type: "nll" (full probabilistic prediction)
- First term: Penalizes return prediction errors, scaled by predicted uncertainty
- Second term: Log-variance regularization prevents over-confidence

NLL loss enables proper probabilistic predictions by jointly optimizing return forecasts and uncertainty estimates. The λ_μ=1 setting balances magnitude accuracy with well-calibrated uncertainty, which is critical for downstream mean-variance optimization.

This loss encourages the model to:

1. Predict return magnitudes accurately (not just rankings)
2. Estimate uncertainty honestly (avoids predicting zero volatility)
3. Provide well-calibrated probabilistic forecasts for portfolio optimization

Checkpoint Selection: **Best eval loss**

- Rationale: Portfolio optimization requires accurate magnitudes, not just rankings
- IC measures rank correlation only, discards magnitude information
- Empirical validation: Loss-selected models achieve higher Sharpe ratios
- Best checkpoint restored after training completes

Hyperparameters:

- Learning rate: 2^-6 (0.015625)
- Batch size: 2^11 = 2048 (balances stability and training speed)
- Epochs: 16 with checkpoint selection on best eval loss
- Regularization: L2 weight decay 2^-20, dropout 0
- Attention heads: 2
- Hidden units: 16
- Optimizer: SGD with weight decay

Training Metrics:

- **Train Loss**: NLL loss on training set (decreases during training)
- **Eval Loss**: NLL loss on evaluation set (used for checkpoint selection)
- **IC Calculation Removed**: Not computed during training for efficiency

Prediction Quality:

- Information Coefficient: 0.0808 on test set (2024-2025)
- Eval loss-based selection improves portfolio translation over IC-based selection
- Key insight: Magnitude accuracy matters more than rank correlation for MVO

- Feature timescale alignment (20-day features for 20-day predictions) improved generalization

The joint return-volatility prediction is critical for portfolio optimization, as it enables both expected return maximization and explicit risk control.

Dataset Construction

Universe: Russell 1000 stocks (~600-700 tickers depending on period)

Data Sources:

- **Fundamentals**: SimFin quarterly financial statements
- **Prices**: Daily OHLCV data from Yahoo Finance
- **Market Data**: S&P 500, Treasury yields, VIX futures

Dataset Generation (`features/dataset.py`):

1. **Feature Calculation**: Compute all technical, fundamental, and market features
2. **Temporal Alignment**: Align features to common trading dates
3. **Window Creation**: Create 60-day lookback windows for each prediction point
4. **Target Calculation**: Compute 20-day forward returns and realized volatility
5. **Validation**: Remove samples with insufficient history or missing targets

Data Structure:

- **Features (X)**: (samples, 60 days, 41 features) tensor
- **Targets (y)**: (samples, 2) tensor [forward return, forward volatility]
- **Metadata**: Ticker symbols, dates, market caps for each sample

Sample Counts:

- Training: ~1.29M samples (2010-2021, dataset_20260120_223016)
- Evaluation: ~250K samples, 649 tickers (2022-mid 2023)
- Testing: ~186K samples, 749 tickers (mid 2024-mid 2025)

Storage: Pickled FeaturesDataset objects (~4.4 GB each) for fast loading

Portfolio Optimization

The portfolio construction module (`portfolio/`) translates model predictions into optimal position weights using mean-variance optimization with explicit constraints.

Mean-Variance Optimization (`portfolio/model.py`):

Objective: Maximize risk-adjusted returns

```
max w^T μ - λ w^T Σ w
```

Where:

- w: Portfolio weights (position sizes)
- μ: Predicted returns from forward model
- Σ: Covariance matrix (mixed forward + historical)

- λ: Risk aversion parameter (implicitly determined by constraints)

Market Neutral Constraint:

```
1^T w = 0  (sum of weights equals zero)
```

This ensures the portfolio is dollar-neutral (equal long and short exposure), eliminating market beta and focusing returns on stock selection skill.

Solution (closed-form):

```
λ = (1^T Σ^-1 μ) / (1^T Σ^-1 1)
w = Σ^-1 (μ - λ1)
```

This computes optimal weights that maximize the Sharpe ratio subject to market neutrality.

Covariance Estimation (`portfolio/covariance.py`):

Mixed Covariance Approach:

```
Σ = α × Σ_forward + (1-α) × Σ_historical
```

Where:

- α = 0.5: Mixing parameter (50% forward, 50% historical)
- Σ_forward: Historical correlations scaled by predicted volatilities
- Σ_historical: Sample covariance with Ledoit-Wolf shrinkage

Historical Covariance:

- Window: 250 trading days (~1 year)
- Shrinkage: 0.25 (25% toward diagonal for better correlation capture)
- Handles 700+ stocks with robust estimation

Forward Covariance:

1. Extract historical correlation matrix from Σ_historical
2. Scale correlations by predicted volatilities ($\hat{\sigma}$) from forward model
3. Result: Σ_forward = D × ρ_historical × D, where D is diagonal matrix of predicted $\hat{\sigma}$
4. Preserves historical correlation structure but updates volatility forecasts

This mixed approach combines:

- Forward model's ability to predict changing individual stock volatilities
- Historical correlation structure (stable over short horizons)
- Shrinkage to prevent overfitting with high-dimensional covariance
- **Key insight**: Correlations change slowly, but volatilities change rapidly - this captures both

Position Sizing Constraints (`config/hyperparameters.py`):

- **min_scalar**: 0.25 (2^-2) minimum position size scalar
  - Prevents dust positions with high turnover
- **max_scalar**: 16 (2^4) maximum position size scalar
  - Allows aggressive conviction sizing on highest-signal stocks

    - With optimal weights, results in moderate gross leverage
- **max_leverage**: 2 (2^1) total gross exposure limit
    - Sum of absolute weights ≤ 2
    - Conservative leverage constraint for risk control
    - Limits portfolio to 2x total exposure
- **risk_free_rate**: 0.005 (0.5% for Sharpe ratio calculation)
    - Reflects low-rate environment and realistic borrowing costs

Rebalancing:

- Frequency: Every 20 trading days (~4 weeks)
- Rationale: Balances alpha capture vs. transaction costs
- Universe: 600-700 positions (nearly full Russell 1000)

Transaction Costs (`portfolio/portfolio.py`):

- **Slippage**: 5 basis points (0.05%)
    - Applied via price multiplier at entry
- **Commission**: 2 basis points (0.02%)
    - Applied on dollar value of trades
- **Total**: ~7 bps one-way, ~12-14 bps round-trip
- **Annual Impact**: ~0.6-1.5% drag depending on turnover

Portfolio Simulation:

1. Generate predictions for all stocks on rebalancing date
2. Estimate covariance matrix (mixed approach)
3. Solve for optimal weights with market neutral constraint
4. Apply position size limits and leverage constraint
5. Execute trades with transaction costs
6. Track P&L, drawdown, Sharpe ratio daily
7. Repeat on next rebalancing date

Key Design Insights

Checkpoint Selection: Models are selected by **lowest eval loss**. Portfolio optimization requires accurate return magnitudes, not just correct rankings. Empirical results show loss-selected models achieve significantly higher Sharpe ratios.

Feature Timescale Alignment: Using 20/60/120/250-day feature windows (aligned with 20-day prediction horizon) improves generalization over mixed timescales like 5/60/120/250-day. This alignment improved model generalization while reducing architecture to 16 hidden units and 2 attention heads (~3.8K parameters).

Loss Function: The model uses NLL (Negative Log-Likelihood) loss with lambda_mu=1 for balanced return prediction and uncertainty calibration. This provides well-calibrated probabilistic forecasts essential for mean-variance optimization.

Observed Trade-offs:

- **Checkpoint selection criterion**: Eval loss (magnitude accuracy) > IC (rank correlation) for portfolio optimization
- **Feature timescales**: Aligning feature windows with prediction horizon improves generalization

- **Conservative leverage**: max_leverage=2x with max_scalar=16 → 4.46% annualized returns with 2.75% drawdowns (risk-controlled strategy)
- Less frequent rebalancing (20 vs 16 days) → Lower transaction costs, higher net returns
- Lower covariance shrinkage (0.25 vs 0.5) → Better correlation capture for diversification
- NLL loss with lambda_mu=1 → Balanced return prediction and uncertainty calibration
- Compact architecture (16 hidden units, 2 heads) → ~3.8K parameters prevents overfitting
- Risk-free rate in Sharpe: 0.5% reflects low-rate environment

Project Structure

```
transformer-portfolio/
│
├── config/
│   ├── hyperparameters.py       # All model and portfolio parameters
│   └── paths.py                 # Data paths configuration
│
├── data/
│   ├── raw/
│   │   ├── loader.py            # SimFin data loading
│   │   └── simfin/              # Fundamental data CSVs
│   └── datasets/                # Generated FeaturesDataset pickles
│
├── features/
│   ├── dataset.py               # FeaturesDataset class and generation
│   └── features.py              # Feature orchestration
│
├── equities/
│   ├── data.py                  # Equity data aggregator
│   ├── features.py              # Equity feature orchestration
│   ├── fundamentals/
│   │   ├── fundamentals.py      # Fundamental data management
│   │   ├── balances.py          # Balance sheet processing
│   │   ├── earnings.py          # Income statement processing
│   │   └── shares.py            # Share count and market cap
│   └── technicals/
│       ├── technicals.py        # Technical indicator management
│       ├── trend.py             # Momentum and ROC
│       ├── relation.py          # Relative returns and beta
│       ├── volatility.py        # ATR and volatility
│       ├── score.py             # Normalized price/volume scores
│       └── ...
│
├── markets/
│   ├── data.py                  # Market data aggregator
│   ├── features.py              # Market feature orchestration
│   ├── observations.py          # Observable market features
│   ├── regime/
```

```
│   │   ├── model.py            # Regime detection pipeline
│   │   └── hmm.py              # HMM implementation
│   └── macros/
│       ├── index_momentum.py   # Market return features
│       ├── index_volatility.py # Market volatility features
│       ├── treasury_slope.py   # Yield curve features
│       └── vix_term.py         # VIX futures features
│
├── models/
│   ├── model.py                # ForwardModel transformer architecture
│   └── models/                 # Saved trained models
│
├── portfolio/
│   ├── portfolio.py            # Portfolio simulation
│   ├── model.py                # MVO optimization
│   ├── covariance.py           # Covariance estimation
│   ├── universe.py             # Universe selection
│   └── positions/
│       ├── position.py         # Position tracking
│       └── entry.py            # Entry price with slippage
│
├── scripts/
│   ├── train.py                # Model training script
│   └── ...                     # Various analysis scripts
│
├── main.py                     # Main entry point
├── analyze_model.py            # Model evaluation script
└── playground.py               # Experimentation notebook
```

Key Design Principles

1. **Modularity**: Each component (features, HMM, forward model, portfolio) is independent
2. **No Look-Ahead Bias**: Strict temporal ordering ensures all features use only past data
3. **Efficient Caching**: Datasets pre-computed and stored for fast iteration
4. **Configurable**: All hyperparameters centralized in config/hyperparameters.py
5. **Extensible**: New features, models, or portfolio strategies easily added
6. **Type Safety**: Type hints throughout for code clarity
7. **GPU Support**: Automatic CUDA utilization for model training

Data Flow Summary

```
Raw Data (SimFin, Yahoo Finance)
         ↓
Feature Engineering (Technical + Fundamental + Market)
         ↓
HMM Regime Detection → Regime Probabilities
         ↓
FeaturesDataset (60-day windows with targets)
         ↓
```

```
Forward Model Training (Transformer + NLL Loss)
         ↓
Predictions (Returns + Volatilities)
         ↓
Covariance Estimation (Mixed Forward + Historical)
         ↓
Mean-Variance Optimization (Market Neutral)
         ↓
Portfolio Simulation (With Transaction Costs)
         ↓
Performance Metrics (IC, Sharpe, Drawdown)
```

Conclusion

This system demonstrates a complete quantitative portfolio management pipeline, from raw data to backtested performance. Key achievements:

- **Strong Predictive Power**: IC 0.08 validates model signal quality
- **Sophisticated Architecture**: Transformer attention mechanism with joint return-volatility prediction
- **Robust Risk Management**: Market neutral with explicit covariance estimation
- **Production-Ready**: Transaction costs, rebalancing, position limits all modeled
- **Extensible Framework**: Modular design supports rapid experimentation

The project showcases expertise in:

- Machine learning for finance (transformers, attention mechanisms, NLL loss)
- Portfolio theory (mean-variance optimization, market neutrality, covariance estimation)
- Feature engineering (technical, fundamental, regime-based)
- Production ML considerations (data leakage prevention, transaction costs, backtesting)

Future enhancements could include: ensemble models, alternative universes (Russell 2000, international), sector neutrality constraints, alternative optimization methods (risk parity, CVaR), and real-time execution capabilities.